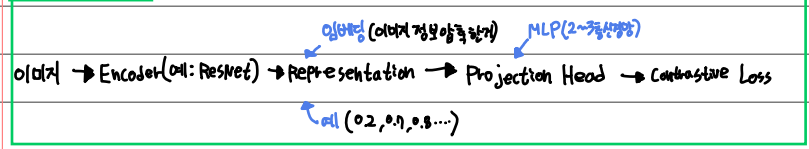


A Simple Framework for Contrastive Learning of Visual Representation (SimCLR)

SimCLR 기본 구조



저작물의 특징

- SimCLR에서 원본 이미지를 다양한 방식으로 변형해서 서로 다른 View를 만들고, 두 View가 같은 것을 예측하는 contrastive task로 설정했다. (대조기반 예측)
- 그냥 평범하게 임베딩 → Contrastive Loss를 하게 않고 MLP (2~3층 신경망)을 만들어서 학습하였다 (Projection Head)
- Contrastive learning이 잘 되려면 큰 batch size와 train epoch가 필요하다.

요약

- 저자들은 discriminative approach에 Contrastive learning을 적용한 간단한 framework인 SimCLR 제안
 - ↳ 쉽게 말하면 한개의 이미지를 가지고 crop, color jitter 등으로 이미지를 변형 한 다음 두개의 이미지끼리 같은지 틀린지 학습시켰다.
- 저자들은 위 에지점 임베딩 벡터를 가지고 Contrastive learning을 적용한 모델로 SOTA 달성했다.

리벨 없이 학습하는 방법

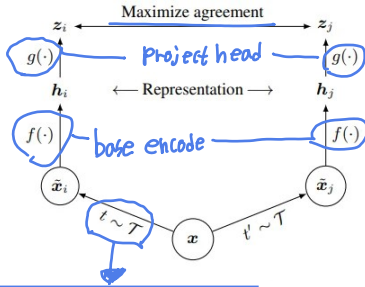
- ① Generative approach
 - U-Net처럼 이미지를 압축시켰다가 다시 원복시켜
 - Pixel-level Generation의 연산 Cost가 높다.
 - ↳ 한 픽셀 한 픽셀 연산하기 때문에 Cost가 크다.
 - ② Discriminative approach
 - CNN으로 이미지에서 임베딩 벡터만 뽑아서 같은건지 다른건지 비교하면서 학습한다.
 - 문제는 Pre-text task를 정의하는 과정에서 heuristic에 의존하게 된다.
 - ↳ 특정된 학습 목표는 상정해본 것
 - ↳ 정확히 최적의 방법의 문제물을 찾는 것이 아닌
- ④ augmentation 확대 회전 주기, 색 변화 가장 간단한 방법으로 문제를 해결해버리는 모경이 생긴다는 것이다.

실험 결과로 부터 도출

- Contrastive Prediction task를 정의함에 있어서, multiple data augmentation이 중요한 역할을 한다.
- Representation과 Contrastive loss에 Projection head (MLP)를 추가함으로써 Representation을 더 잘 학습할 수 있다.
- Contrastive cross entropy loss 사용시, normalized embedding과 temperature parameter가 중요한 역할을 한다.
 - ↳ 임베딩 벡터를 길이 1로 벡터로 정규화
 - ↳ 유사도 score 계산에 사용되는 파라미터
- Contrastive learning은 일반적으로 저도 학습보다 큰 batch size와 긴 train time 필요하다.

과정

Contrastive loss function



Data augmentation module

- 이미지 2개의 augmentation을 각각 적용하여 그 결과를 positive pair로 간주한다.
- 적용되는 augmentation은 크게 3개이다.

① Random crop → Resize back to origin size

② Random color distortion

→ 이미지의 색상정보(Hue)를 무작위로 변경하여 원본 이미지의 구조(모양)은 유지하면서 색상만 다르게 만드는 aug 기법
보통 색상에 의존하지 않고 형태나 패턴에 집중하도록 유도하는데 사용한다.

③ Random Gaussian blur

→ 가우스 커널을 이용해서 랜덤하게 부드럽게 흐리게 만든다.

이미지의 자세한 디테일(경계, 잡음)을 일부러 흐리게 만들어 세부 정보가 아닌 전체 구조, 윤곽에 집중시키기 위함

Base encoder

- Aug img로부터 임베딩 벡터를 추출하기 위한 network로 network의 종류에 구애받지 않는다.
- ↳ 이것저것 시도 상관없고 아까운 ResNet을 사용한다.

Projection head

- Representation vector를 Contrastive loss가 적용되는 차원으로 mapping 하는 network
- MLP 구조는 Linear - BN - ReLU - Linear - BN 구조의 MLP를 사용한다.

Contrastive loss function

- N개의 mini-batch에 aug을 각각 적용하여 2N개를 얻는다.
- 하나의 이미지로부터 얻어진 aug img 2개를 Positive pair로 간주하고, 이외의 이미지 2N-2개를 Negative로 간주한다.
- 수식은 $\frac{1}{2} \|z_i - z_j\|^2$ (정식)

세부적인 작업

batch size

- memory bank를 사용하는 대신 batch size를 8192까지 늘렸다.
- 큰 batch size에서의 안정적인 학습을 하기 위해 LARS optimizer를 사용했다.

memory bank

- GPU 메모리가 부족한 상황에서 Negative 샘플을 더 만들기 위해 과거의 이미지 임베딩을 저장해서 학습할때 사용한다.
- 무제한 가중치가 업데이트 되면 학습도 과거 데이터는 비효율적이다.

batch 8192 와 LARS 옵티마이저

- Memory bank 대체로 Negative 데이터가 많이 필요하기때문에 배치를 키워서 사용했다.
- 군데이전식으로 배치를 키워게 되면 모델의 일반화능력이 감소한다.
- 일반적으로 batch size에 비례해서 LR를 키워야 안정적인데 이렇게 크게하면 가중치가 발산해버린다.
↳ 그래서 Layer 단위로 LR를 조절하는 LARS를 사용하게된다.

이때 weight와 gradient를 고려한다.

$$\Delta W = -\eta \cdot \text{learning ratio} \cdot \nabla W$$

$$\rightarrow \frac{\|W\|}{\|W\| + \epsilon} \quad (\text{간단 버전})$$

Global BN

- Information leakage를 막기 위해, 분산 학습과정에서 모든 device에 대한 BN의 평균과 분산을 한번에 업데이트한다.

→ SimCLR은 오리지널 이미지로 $A_1, A_2, B_1, B_2, C_1, C_2$ 로 나눠서 학습한다.

그러다보니 각 객체의 BN 평균과 분산이 저절로 학습된다.

그래서 배치를 최대한 늘려서 전체적으로 데이터를 모아 BN이 하나의 객체만 치환되지 않도록 한다.

추가적인 실험 결과

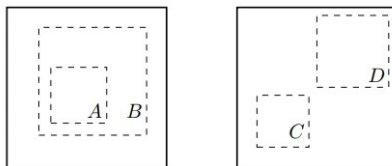
결과) Random Crop과 같은 augmentation을 통해 Global and local views와 Adjacent View 작업을 할수있다.

Global and local views

- 전체 이미지를 보고 그 안의 일부분을 예측하도록 학습하는것
- 일반적으로 CNN같은 경우 conv를 많이 겹칠 수록 작은 객체를 잘 잡아낸다. → 아키텍처 수정
- SimCLR에서는 아키텍처 수정 없이 Crop과 blur, color jitter 등을 이용해서 Global and local views 작업을 수행할수 있다고 한다.

Adjacent views

- 이미지의 일부분만 보고 나머지 부분을 예측하도록 학습하는것
- 일반적으로 Context augmentation network Image Patch에 대한 많은 부분을 예측을 위한 한다.
- SimCLR은 이런부분도 아키텍처 수정 없이 작업을 수행할수 있다고 한다.



(a) Global and local views.

(b) Adjacent views.

Figure 3. Solid rectangles are images, dashed rectangles are random crops. By randomly cropping images, we sample contrastive prediction tasks that include global to local view ($B \rightarrow A$) or adjacent view ($D \rightarrow C$) prediction.

Augmentation

- 논문에서 사용하는 증강

Spatial/geometric transformation

- Crop
- Resize (with horizontal flip)
- Rotation
- Cutout

Appearance transformation

- Color distortion (color drop, brightness, contrast, saturation, hue)
- Gaussian blur
- Sobel filtering

- Aug의 성능을 평가하기 위해 Linear evaluation 방식으로 실험을 진행하였다.

↳ 인코더 부분(CNN 부분)을 고정시키고 뒷단만 학습시킨다.

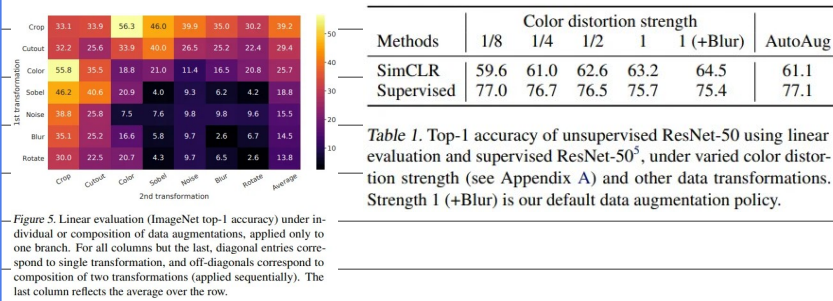
- 실험을 통해 하나의 Aug만 했을 때 성능이 좋지 않았다.

- 대신에 2개의 Aug를 각각 적용하여 어렵게 만들었을 때 성능이 매우 좋아졌다.

- 또한 Color distortion strength의 비율을 늘리는 실험에서는 저도학습보다 SimCLR 성능 향상폭이 커지는 것을 볼 수 있었다.

- 반대로 AutoAug에서는 저도학습이 더 성능 향상이 큰 것을 볼 수 있었다.

- **내생각** Aug는 데이터의 타입, 모델구조, 학습 방법에 따라 전혀 다른 것을 보여주는 것 같다.



모델 크기

- 저도학습과 마찬가지로 대도학습 모델의 depth와 width이 성능에 비례한다.

- m의 크기가 커질수록 저도학습에 비해 성능 향상폭이 증가하는 것을 볼 수 있다.

- 저도학습에는 epoch를 더 키워거나 더 강한 Aug를 적용하더라도 일관된 유의미한 결과는 없었다.

